CIS 5270: Final Project Report

Team: Ricky Gong, Sunggun Lee

1 Introduction

We study a failure-aware reasoning task based on the ARC-AGI-1 dataset. Each task consists of several input/output grid demonstrations and a held-out test input. The model must infer the hidden transformation rule and output the correct test output grid. Each grid is represented as rows of space-separated integers, where each integer corresponds to a color.

In addition to standard ARC tasks, we introduce a refusal-style extension. We synthetically corrupt some ARC tasks so that the examples no longer consistently define a single transformation rule. For these corrupted tasks, the desired output is the special token `REFUSE`. This lets us evaluate both rule-inference accuracy on valid tasks and failure-aware behavior on inconsistent tasks. Our experiments compare the base `gpt-4.1-nano` model against supervised fine-tuned variants and a DPO-tuned model.

2 Data

We used the official ARC-AGI-1 tasks dataset and converted each task into a text-based chat format. For clean tasks, the prompt contains the original training demonstrations followed by a test input, and the target response is the correct output grid. For corrupted tasks, the prompt contains an intentionally inconsistent set of demonstrations, and the target response is `REFUSE`.

Split	Clean Tasks	Corrupted Tasks	Total
Train	330	51	381
Validation	70	13	83
Test	400	85	485

Table 1: Dataset split sizes for clean and corrupted ARC-style tasks

Within each split, tasks were selected for corruption independently with probability 20%. For each selected task, we randomly chose two demonstrations and replaced the output of one demonstration with the output of the other. The resulting prompt therefore contained one inconsistent input-output pair, and its target response was set to `REFUSE`. Clean tasks were preserved rather than replaced, so corrupted tasks were added alongside the original clean tasks. Each corrupted task was derived from a clean task within the same data split, ensuring no cross-split contamination.

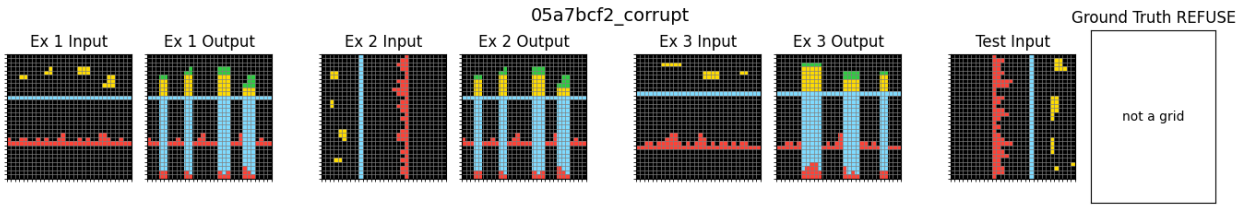


Figure 1: Current One-Replace Corruption Strategy: Ex 2 Output Replaced with Ex 1 Output

3 Base Model

3.1 Prompt format

For the base model, we used a chat-style prompt with a system instruction and a user message. The system instruction was:

> `You solve ARC-style grid transformation tasks. Infer the transformation rule from the examples. Output only the final answer grid as rows of space-separated integers. If the demonstrations are contradictory and no consistent rule exists, output exactly REFUSE.`

The user message listed demonstrations as labeled input-output pairs: `Example 1 Input` → `Output`; `Example 2 Input` → `Output`; ... followed by `Test Input`. The model was expected to output only the final answer grid or exactly `REFUSE`.

3.2 Inference configuration

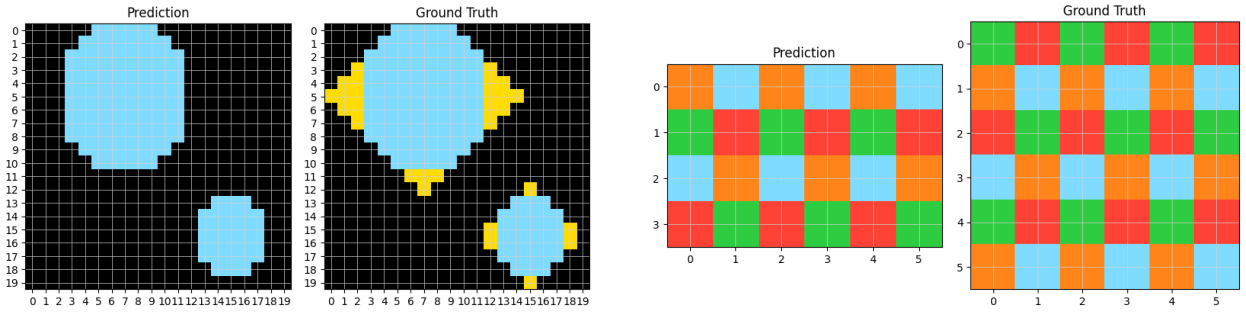
The base model was evaluated using `gpt-4.1-nano` with temperature 0.0 and a maximum completion length of 800 tokens. This choice ensures deterministic decoding with consistent outputs across runs, isolating the model's inherent ability without introducing sampling variability. The maximum completion length was chosen to accommodate full grid outputs without truncation. All evaluations were performed by running the model once per prompt without retries or filtering.

3.3 Base model results

Performance was evaluated using exact-match accuracy (EM) on clean ARC tasks, average cell-level accuracy (CA) on clean tasks, and refusal accuracy (RA) on corrupted tasks. Let $\hat{y}$ denote the model prediction and y the ground truth, and where H and W are the grid height and width.

$$\mathrm{EM}(\hat{y}, y) = \mathbf{1}\{\hat{y} = y\}, \quad \mathrm{CA}(\hat{y}, y) = \frac{1}{HW} \sum_{i,j} \mathbf{1}\{\hat{y}_{ij} = y_{ij}\}, \quad \mathrm{RA}(\hat{y}) = \mathbf{1}\{\hat{y} = \texttt{REFUSE}\}. \tag{1}$$

The base model struggled on clean ARC tasks, achieving no exact matches on the held-out clean test set. However, average cell accuracy was nonzero, indicating that some outputs partially matched the ground-truth grid structure. On corrupted tasks, the base model sometimes produced `REFUSE`, but this should not be over-interpreted as reliable abstention behavior.



(a) Good prediction (TASK: 9772c176), CA: 0.93

(b) Poor prediction (TASK: 00576224), CA: 0.0

Figure 2: Example of Baseline Predictions on Sample ARC tasks

Model	Exact Match (EM)	Cell Accuracy (CA)	Refusal Accuracy (RA)
Base `gpt-4.1-nano`	0.000	0.073	0.282

Table 2: Baseline performance on the test split

Qualitatively, the base model often produced outputs with incorrect grid size, invalid formatting, or plausible-looking but incorrect transformations. In some cases, the model ignored the output constraints entirely and generated free-form explanations instead of a single grid. The model also output `REFUSE` on 136 out of 400 clean test tasks, corresponding to a 34% false-refusal rate. These behaviors suggest that the raw model does not reliably infer ARC-style transformations or genuinely detect contradictory demonstrations.

4 Supervised Fine-Tuning

4.1 Training data generation

For SFT, we constructed training examples by converting ARC tasks into a chat-format JSONL dataset. Each example consists of the same system message used in the baseline, a user message containing ARC examples and a test input, and an assistant message containing the target output.

For clean tasks, the target output is the ground-truth ARC output grid. For corrupted tasks, the target output is `REFUSE`. Corrupted examples were generated as described in Section 2. This construction allows the model to learn both the required output format and the task behavior: producing a grid for consistent ARC tasks and abstaining when the demonstrations are inconsistent.

4.2 Training configuration

We fine-tuned `gpt-4.1-nano-2025-04-14` using the Azure OpenAI supervised fine-tuning API. Each example used the same chat-style prompt format as in the baseline evaluation. For clean tasks, the assistant target was the gold ARC output grid, while for corrupted tasks the target was the token `REFUSE`.

The training dataset consisted of 381 examples (330 clean and 51 corrupted), with an additional 83 validation examples (70 clean and 13 corrupted). We used supervised fine-tuning with a single epoch, batch size of 1, and learning rate multiplier of 1.0 (default settings).

The SFT objective is standard token-level negative log-likelihood:

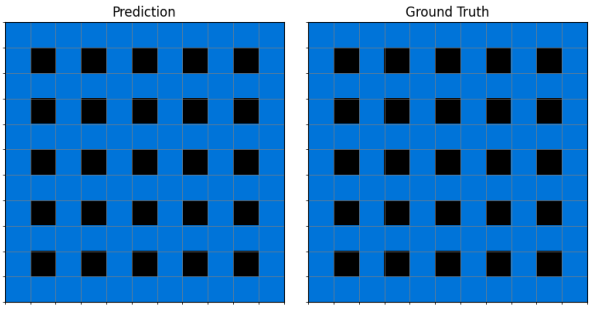
$$L_{\mathrm{SFT}}(\theta) = -\frac{1}{N}\sum_{i=1}^{N}\sum_{t=1}^{T_i}\log \pi_\theta(y_{i,t} \mid x_i, y_{i,<t}),$$

where N is the number of training examples, x_i is the serialized ARC prompt, and $y_i = (y_{i,1}, \ldots, y_{i,T_i})$ is the target assistant response. For clean tasks, y_i corresponds to the gold output grid, while for corrupted tasks y_i is exactly `REFUSE`.

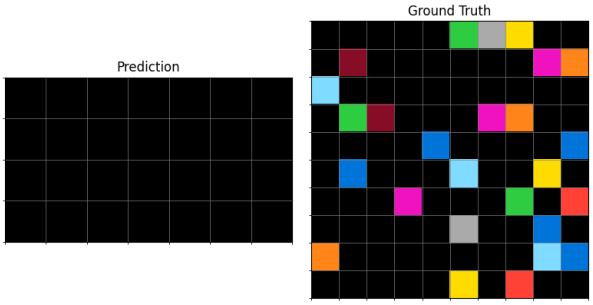
4.3 SFT results

The SFT model substantially improved clean-task cell-level accuracy relative to the base model, indicating that supervised fine-tuning helped the model better learn the required output format and capture aspects of ARC-style grid structure. In addition, SFT produced a small number of

exact matches (two tasks), which were absent in the baseline. However, performance on corrupted tasks degraded as refusal accuracy dropped to 0%. This suggests that while SFT improved the model's ability to generate plausible grids, it failed to reliably learn the intended refusal behavior.



(a) Exact Match (TASK: 332efdb3), CA: 1.0



(b) Poor prediction (TASK: 1a6449f1), CA: 0.0

Figure 3: Example of SFT Predictions on Sample ARC tasks

Model	Exact Match (EM)	Cell Accuracy (CA)	Refusal Accuracy (RA)
SFT `gpt-4.1-nano`	0.005	0.380	0.000

Table 3: SFT performance on the same test split.

One likely explanation is that SFT increased the model's confidence in generating outputs, biasing it toward always producing a grid rather than abstaining. For corrupted tasks, it appears that SFT resorts to mimicking or copying demonstration outputs. Another contributing factor may be the corruption strategy. Our original approach replaced a single demonstration output with another, which may not have been sufficiently adversarial. Fully permuting input-output pairings may more reliably break task consistency.

To test these hypotheses, we ran two additional SFT variants. First, we tested whether refusal behavior could be recovered by balancing the dataset. Instead of replacing clean examples, we kept all 330 clean training examples and added 330 corrupted variants, yielding a 50/50 clean-corrupt training split. The validation set was similarly balanced with 70 clean and 70 corrupted examples. Second, we compared two corruption strategies: the original single-output replacement method and a stronger permutation-based method that shuffled the demonstration outputs across examples.

SFT Variant	EM	CA	RA
Original SFT	0.005	0.380	0.000
Balanced SFT, old corruption	0.000	0.000	1.000
Balanced SFT, permutation corruption	0.000	0.000	1.000

Table 4: SFT performance across original and balanced variants

The balanced SFT variants recovered refusal behavior, achieving perfect refusal accuracy on corrupted tasks, but collapsed in the opposite direction: both models failed to solve clean tasks, producing zero exact-match accuracy and zero cell-level accuracy. This suggests that the model is highly sensitive to the relative strength of the solving and refusal signals. With mostly clean data, SFT learns to generate grids even when it should abstain; with balanced clean-corrupt data, it learns that `REFUSE` is a globally safe response. These results indicate a tradeoff between reasoning and abstention that may need further experimentation to properly obtain both capabilities.

5 Direct Preference Optimization (DPO)

5.1 Preference data generation

To better align the model's behavior between solving valid tasks and abstaining on inconsistent ones, we applied Direct Preference Optimization (DPO) on top of the SFT model. DPO requires pairs of preferred and non-preferred outputs for the same input prompt.

Initial DPO experiments revealed two key challenges: (1) collapse to `REFUSE` behavior across both clean and corrupted tasks, and (2) loss of training signal due to weak preference pairs. To address these issues, we designed a more structured preference data generation strategy.

For each prompt, we sampled four candidate outputs from the SFT model. For clean tasks, preference pairs were constructed using cell-level accuracy (CA) as a quality metric. When a sufficiently large gap existed between sampled outputs, we selected the highest-CA output as the preferred response and the lowest-CA output as the rejected response. When sampled outputs were too similar, we used the ground-truth output grid as the preferred response to ensure a strong preference signal. For rejected outputs on clean tasks, we prioritized `REFUSE` if it appeared among the sampled outputs. Otherwise, we selected either invalid outputs or the lowest-CA sampled grid. Pairs were discarded only when the preferred and rejected outputs were identical.

For corrupted tasks, preference construction was more direct. Since the correct behavior is to abstain, `REFUSE` was always treated as the preferred output, while any generated grid was treated as the rejected output. Corrupted-task pairs were discarded only when all sampled outputs were `REFUSE`, resulting in no valid negative example.

Split	Total Pairs	Clean Pairs	Corrupted Pairs
Train	380	329	51
Validation	83	70	13

Table 5: Number of DPO preference pairs for clean and corrupted tasks.

5.2 Training configuration

We trained the DPO model using the Azure OpenAI DPO pipeline with `gpt-4.1-nano-2025-04-14`. Preference pairs were generated by sampling multiple outputs from the SFT model, and DPO training used the same chat-style ARC prompt format as SFT. Due to budget and deployment constraints, we ran one DPO configuration rather than performing an extensive hyperparameter sweep. The DPO run used one epoch, batch size 1, learning rate multiplier 1.0, and $\beta = 0.1$.

DPO can be viewed as optimizing a preference objective that increases the likelihood of preferred outputs y^+ relative to rejected outputs y^- for the same prompt x. In simplified form:

$$L_{\mathrm{DPO}}(\theta) = -\mathbb{E}_{(x,y^+,y^-)}\left[\log \sigma\left(\beta\left(\log \pi_\theta(y^+|x) - \log \pi_\theta(y^-|x)\right)\right)\right],$$

where y^+ and y^- denote preferred and rejected outputs, respectively, and β controls the strength of the preference update.

5.3 DPO results

Despite the improved preference pair generation strategy, the DPO model failed to balance task-solving and refusal behavior. While the model achieved perfect refusal accuracy (RA = 1.0) on

corrupted tasks, this result reflects a degenerate policy rather than successful failure-aware reasoning. In particular, the model defaulted to outputting `REFUSE` across both corrupted and clean tasks, resulting in zero exact-match accuracy and negligible cell-level accuracy on clean ARC tasks.

Model	Exact Match (EM)	Cell Accuracy (CA)	Refusal Accuracy (RA)
DPO (improved)	0.000	0.000	1.000

Table 6: DPO performance on the test split

Although our improved pair generation strategy attempted to mitigate this by including `REFUSE` as a rejected output for clean tasks when available, this signal was insufficient. In practice, `REFUSE` appeared relatively infrequently among rejected outputs in clean-task pairs, resulting in a weak penalty for over-refusal. As a result, the model was not adequately discouraged from choosing `REFUSE` in situations where a valid solution exists.

5.4 Limitations and future improvements

The observed failure mode highlights the tension between solving ARC tasks and detecting inconsistency. The model's behavior is highly sensitive to the training signal, which can push it toward either always generating outputs or always abstaining.

Because `REFUSE` is always the correct response for corrupted tasks and is only weakly penalized in clean-task preference construction, the model learns that abstention is a globally safe strategy. This leads to a collapse toward trivial refusal behavior. More broadly, our experiments suggest there exists a tradeoff between task-solving performance and refusal accuracy, where improvements in one objective can come at the expense of the other.

Specifically for DPO, several improvements could address these issues. First, preference construction should explicitly penalize refusal on clean tasks by including `REFUSE` as a rejected output, even when it is not sampled. This would discourage trivial refusal policies and enforce a stronger distinction between solvable and inconsistent tasks. Second, sampling a higher quantity and diversity of SFT outputs for a task could enforce stronger preference gaps.

Due to budget and time constraints, we were unable to iterate further on DPO training. However, the observed behaviors already illustrate the difficulty of balancing reasoning and abstention using standard preference-based objectives.

6 Conclusion

In this work, we studied a failure-aware extension of ARC-style reasoning, where models must both infer transformation rules for valid tasks and abstain when demonstrations are inconsistent. We found that the base `gpt-4.1-nano` model performs poorly on exact ARC reasoning and exhibits unreliable refusal behavior. Although the base model sometimes outputs `REFUSE` on corrupted tasks, it also frequently refuses clean tasks, suggesting that its refusal behavior reflects uncertainty rather than calibrated inconsistency detection.

Supervised fine-tuning improved clean-task output quality and cell-level accuracy, but eliminated refusal behavior entirely. This indicates that standard SFT can teach the model to produce more plausible grid outputs, but may also bias it toward always answering even when the task is inconsistent. Our balanced SFT experiments showed the opposite failure mode: increasing the proportion of corrupted examples recovered refusal accuracy but caused the model to collapse toward always outputting `REFUSE`. Similarly, DPO did not resolve this tension. Despite improved

preference-pair construction, the model still over-preferred refusal, likely because `REFUSE` was not penalized strongly enough on clean tasks.

Overall, our experiments reveal a sensitive tradeoff between task-solving and abstention in failure-aware structured prediction. Standard SFT and naive DPO preference construction were not sufficient to produce calibrated behavior across both clean and corrupted ARC tasks. Future work should improve corruption generation, explicitly penalize refusal on clean tasks, and construct stronger preference pairs that better separate solvable tasks from truly inconsistent ones. These results suggest that failure-aware reasoning requires carefully balanced training signals, rather than simply adding refusal examples to a standard fine-tuning pipeline.